

ICLR 2023 | For Multilingual LLMs, One Temperature Knob Isn't Enough

Google Research proposes UniMax: more uniform budget for high-resource languages, a hard cap on how often low-resource ones repeat

Train a model across a hundred-plus languages and the first unavoidable question is simple: how much data should each language get? Real corpora are wildly imbalanced. Sample in proportion to size and English swamps everything; flatten it and the low-resource languages get their tiny slice repeated over and over. The standard fix is temperature sampling, a hyperparameter τ that interpolates between "proportional" and "uniform."

That compromise is not free. In mC4, English has roughly 9.7 trillion characters, over 92,000 times the size of the lowest-resource language, Yoruba. Raising the temperature to help the head languages over-repeats the tail: at $\tau=3.33$ with a one-trillion-token budget, the lowest-resource languages are seen more than 100 times. And whether that knob is set well, and stays well-set as the model grows, had never been studied in a systematic way.

This paper proposes UniMax (uniform + max): spread the budget as uniformly as possible across languages, while capping how many times any single language may repeat. Head languages get more uniform coverage than under temperature sampling, and tail languages are kept from being repeated dozens of times over. The payoff: across many benchmarks and model sizes, UniMax consistently beats temperature sampling, and the advantage holds as models scale up.

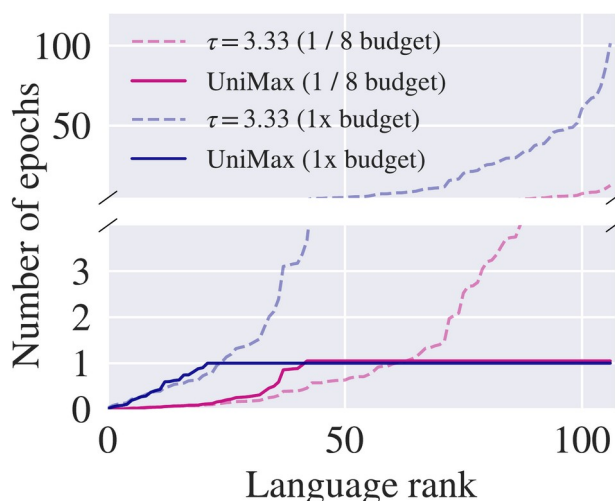


Figure 1. Training epochs per language against the language's resource rank (x-axis sorted by character count; lower-resource languages sit to the right). Temperature sampling (dashed, $\tau=3.33$) repeats the lowest-resource languages over a hundred times, while UniMax (solid) holds every language's repeat count under a set cap (here, one epoch).

Paper: <https://arxiv.org/abs/2304.09151> | Code and checkpoints (T5X): <https://github.com/google-research/t5x>

The problem: imbalance, and no free lunch from temperature

The central tension in multilingual pretraining is that language sizes differ by orders of magnitude. Over-repeating a low-resource language carries three concrete costs: it overfits and drags down downstream tasks; it raises the risk of memorizing and leaking private training content; and it burns compute on duplicate examples. Worse, the paper observes that all three harms grow more severe as models get larger, so a temperature that "looks fine" on a small model may not hold on a big one.

The opposite end has its own failure mode: set the temperature too low, sample the low-resource languages too rarely, and they end up undertrained. The next figure makes that tension visible.

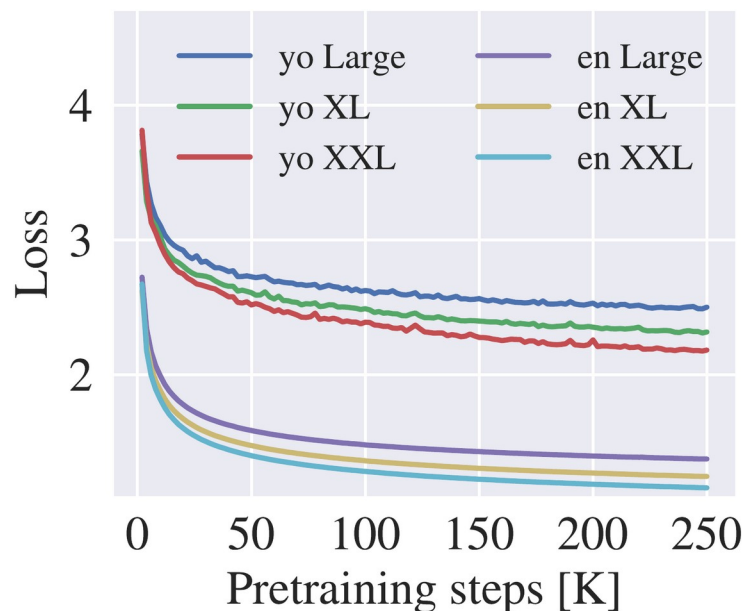


Figure 2. Held-out pretraining cross-entropy loss for a high-resource language (English, en) and a low-resource one (Yoruba, yo) across Large, XL, and XXL sizes. Yoruba sits well above English at every scale: get the balance wrong and low-resource languages are undertrained. The paper also observes that at higher temperatures, overfitting worsens as models grow. (Loss values are not directly comparable across sampling strategies because the vocabularies differ.)

In other words, temperature sampling walks a tightrope between undertraining the tail and overfitting it, and the best place to stand on that rope shifts as the model grows. UniMax's goal is to replace the tightrope with a steadier rule.

The method: UniMax = uniform allocation + a repeat cap

UniMax starts from a fixed character budget C and processes languages from lowest- to highest-resource. At each step it asks one question: if the remaining budget were split uniformly among the languages not yet handled, would any language exceed N epochs over its own corpus (that is, repeat itself more than N times)? If so, that language is capped at N epochs and the freed-up budget is redistributed uniformly among the rest; if not, the split is simply uniform.

As a rule of thumb: each language's allocation U is the smaller of " N times its corpus size" and "the remaining budget divided by the number of languages left," and whatever it takes is subtracted from the running budget. The default $N=1$ means no language's data is ever used more than once. Head languages thus get more uniform coverage than temperature sampling would give them, while tail languages are explicitly protected from being repeated dozens of times. The method has essentially no knob to tune, and drops in wherever temperature sampling was used.

Results: consistent gains across benchmarks and scales

The authors pretrain on a refreshed mC4 and evaluate on a broad multilingual suite: TyDi QA GoldP, WMT21 translation, XNLI, XQuAD, MLQA, and PAWS-X. Model size is varied from Small up to XXL, precisely to test whether the benefit survives scale. Start with reading comprehension on TyDi QA.

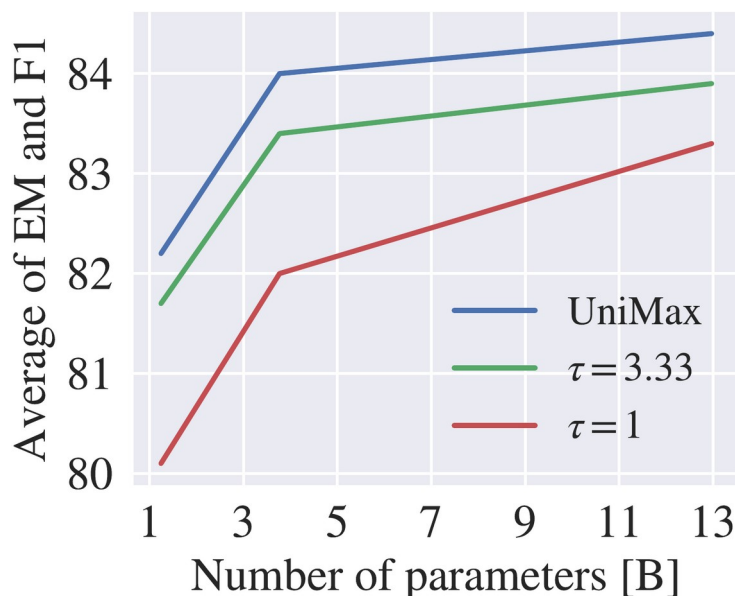


Figure 4. Average TyDi QA GoldP (mean of EM and F1) across three model sizes. UniMax (blue) sits above both $\tau=3.33$ (green) and $\tau=1$ (red) at every scale, and the margin holds as the model grows.

UniMax leads both temperature baselines at every scale. A finer per-language breakdown shows it beats $\tau=3.33$ on both high- and low-resource languages, and only trails $\tau=1$ on high-resource languages at the largest scale, which is intuitive: $\tau=1$ hands the most data to the head languages by design.

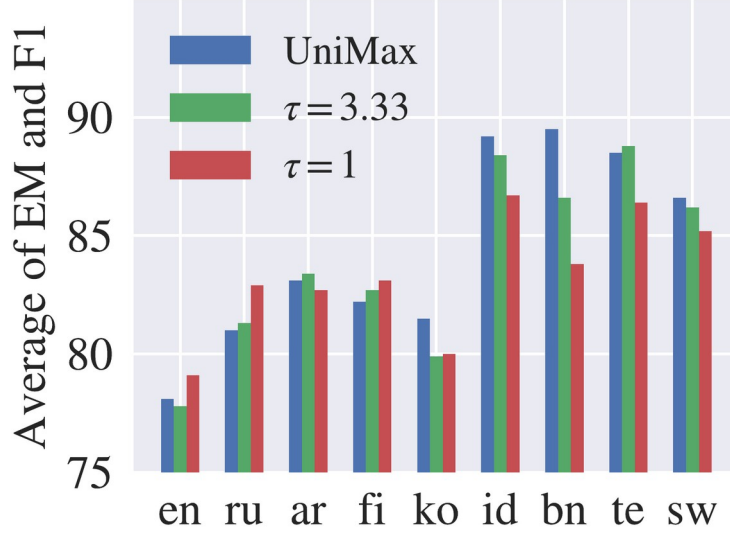


Figure 5. Per-language TyDi QA on the XXL model, with the x-axis running roughly from higher-resource (en) to lower-resource (sw). UniMax’s lead is clearest on low-resource languages (Swahili sw, id, bn, te); it only trails $\tau=1$ on high-resource languages such as English.

Swahili (sw) is a telling case: even though UniMax sees fewer Swahili examples than temperature sampling does, it still beats $\tau=3.33$ on that language. The lesson is that what matters is not how many times you repeat a language, but whether the overall allocation is sound. Translation tells the same story.

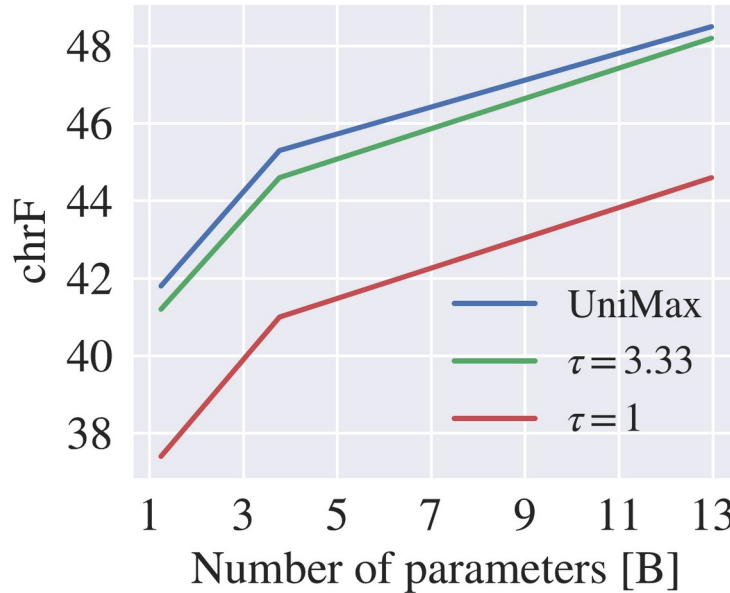


Figure 6. WMT21 multilingual translation, chrF averaged over all language pairs at each scale. UniMax (blue) beats both temperature baselines at every size, and the majority of individual language pairs benefit.

To pin down the most representative numbers: on Large models trained at a 4× larger character budget (which the paper denotes the 1/2 budget), the three strategies land as follows on average TyDi QA.

Table 1. Average TyDi QA for Large models at the 4×-larger (1/2) character budget: UniMax leads both temperature-sampling baselines.

Sampling strategy	TyDi QA (1/2 budget, Large)
UniMax	83.1
$\tau = 3.33$	82.8
$\tau = 1$	81.2

As for what to set the repeat cap N to, the authors ablate it on Large models: $N=1, 5, 10$ give 82.2, 81.5, 81.8 respectively, so disallowing repeats entirely ($N=1$) is actually best, though the gap is small. They also note that the optimal N depends on the character budget, which makes it better read as a robust default than as a knob that needs careful tuning.

Also shipped: a refreshed mC4 and umT5

Beyond the method, the paper ships two things you can use directly. The first is a refreshed, enlarged mC4 corpus: 29 trillion characters across 107 languages, about 35% larger than the original. The second is the umT5 family of checkpoints trained with UniMax. For reference, umT5-XXL reaches 81.2/89.7 (EM/F1) on TyDi QA, ahead of the same-size mT5-XXL at 79.5/88.7.

Takeaway and boundaries

UniMax's claim is a modest one: cap how often each language repeats, spread the rest of the budget as uniformly as you can, and you reliably beat temperature sampling for multilingual pretraining, with the advantage holding as models scale. It needs almost no tuning, drops in as a replacement for temperature sampling, and arrives with a refreshed corpus and umT5 weights. Worth keeping in view: these results come from the mC4 and mT5/umT5 family, with evaluation centered on cross-lingual understanding and translation, and the best repeat cap is tied to the character budget. On a different corpus, model family, or task, the default is still worth re-checking with the same reasoning.